

Bridging the Communication Gap: AI-Powered Smart Gloves for Sign-to-Speech and Speech-to-Text Integration

Abdelrahman Osman ¹ and Ahmed Mohamed ¹

¹ STEM High School for Boys – 6th of October, Giza 12566, Egypt

Abstract

People who are deaf and mute often encounter challenges in everyday interactions with those unfamiliar with sign language. This communication barrier can prevent them from fully expressing themselves and feeling understood. Deaf individuals represent approximately 70 million people, according to the World Federation of the Deaf; thus, addressing this issue is essential for fostering inclusivity. Our project was designed to bridge this communication gap, making interactions between deaf and mute individuals and those who do not know sign language more accessible. We achieved this through two main features: first, our glove can detect and interpret hand movements in sign language, converting them into audible speech. This function enables hearing individuals to understand what a deaf person is communicating without requiring sign language skills. Second, we incorporated speech-to-text capabilities in our glove, allowing it to receive spoken language and display it as text on an LCD screen. This feature enables the deaf person to read what the other person is saying, bridging the gap for two-way communication. In essence, our project provides a practical solution that translates sign language into speech and spoken language into readable text, facilitating smooth, inclusive communication.

Keywords: Assistive communication technology; Sign language recognition; Wearable smart glove; Flex sensor-based gesture detection; ESP32 embedded system; Azure Speech-to-Text (STT)

1. Introduction

Innumerable countries face a significant challenge in providing educational opportunities for people with disabilities, particularly the deaf community. Numerous deaf individuals' parents often lack knowledge of sign language. This creates substantial obstacles for deaf students in communicating with their families and relatives. When parents are unfamiliar with sign language, their children miss essential life lessons and experiences, leading to a communication gap between deaf children and their parents.

Furthermore, sign language is rarely taught in many schools, making it difficult for deaf students who attend mainstream public or private schools to communicate effectively with peers and teachers. This often results in misunderstandings and frustration, leaving deaf students discouraged and unable to fully participate in lessons. Such issues need urgent attention, as they can have long-term consequences on employment rates and social development.

To address this challenge, many governments have established schools specifically for deaf students. This solution has both benefits and limitations. On the positive side, these schools allow deaf students to learn and study alongside their hearing peers, enabling them to pursue higher education and professional careers. However, a significant communication

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the

[Creative Commons Attribution \(CC BY\)](#) license.

gap remains between deaf individuals and their families, relatives, and future employers who often lack sign language skills, leaving this gap unaddressed despite specialized education.

In addition to creating dedicated schools and universities for the deaf, scientists have been developing a tool called “speaking gloves,” which convert sign language into spoken words. These gloves help bridge the gap by allowing people who do not know sign language to understand deaf individuals. However, a major limitation is that this technology only converts sign language to speech. While it allows hearing individuals to understand deaf individuals, it does not enable full two-way communication, as deaf individuals cannot hear responses. Therefore, although innovative, this solution does not fully resolve communication challenges.

Our project aims to bridge this communication gap more effectively. We are developing a glove that converts specific sign language gestures into common, everyday sentences. Additionally, the glove can receive spoken responses from hearing individuals and convert them into text displayed on an LCD screen, allowing deaf individuals to understand responses in real time. This process, enabled by Microsoft Azure’s speech-to-text technology, takes only a few seconds, achieving an efficiency rate of over 90. Our project presents a highly viable solution for the future, promising to improve communication and understanding between deaf individuals and the hearing community.

2. Background & Context

The deaf struggled a lot in many aspects, for example, in their schools, universities, daily life communications, and work. Consequently, multiple inventors tried to ameliorate the deaf by creating numerous projects, such as developing a sign language recognition prototype using a Leap Motion Controller (LMC), proposing a wearable-sensor-based ASL translation system that captures both hand movements and facial expressions, and developing a smart glove system paired with a mobile app to translate sign language gestures into both text and audible speech in real time.

Each one of these projects has points of strength and points of weakness. For instance, recognizing the sign language by using the Leap Motion Controller (LMC). This project was made to recognize the full American Sign Language alphabet (26 letters) and digits 0–9. The system applied machine learning (SVM and deep neural network) to distinguish static signs and those involving motion. Regardless of its high accuracy (approximately 93.8%) and its partial solution to the deaf communication gap, it requires the deaf to use a laptop or computer to detect their gestures and then recognize them. In addition to needing high technological applications, which makes using this technique in the streets impossible, it does not bridge the communication gap between the deaf and people who do not understand sign language, because if anyone wants to say anything to the deaf without using sign language, they will not be able to do that, because this technique only recognizes sign language; thus, it does not help the deaf in their daily life communication.

Additionally, many other projects that used different methods face similar challenges. Therefore, after learning from these projects’ points of strength and improving their points of weakness, our team defined goals: creating a wearable glove to translate sign language to an audible voice without needing a mobile or laptop, and adding a method to make the deaf able to understand hearing people using an API which converts speech to text. Our goal is not only to solve the challenge, but to solve it with accuracy above 90% using accessible materials and technology.

3. Materials

Table 1. Hardware components and their placement, electrical interfaces, and roles in ECO_Hands.

Component	Placement on glove	Electrical connection	Function in ECO_Hands
ESP32 microcontroller (WROVER)	Inside glove (wrist / lower back-of-hand)	ADC1 + I ² C + I ² S + UART + Wi-Fi	Central controller that reads sensors, runs recognition logic, hosts the web settings page, and sends speech segments to Azure.
Flex sensors (×5)	Outside glove on each finger (dorsal side)	Voltage divider → ESP32 ADC1 pins	Measures finger bending to classify static gesture posture. Resistance increases with bend.
MPU6050 (accelerometer/gyroscope)	Under/near LCD (back-of-hand region)	I ² C (SDA/SCL)	Captures wrist/palm motion and orientation signals; calibrated in firmware for future multimodal use.
LCD 16×2 (I ² C)	Outside glove (center back-of-hand)	I ² C	Displays recognized gestures and speech transcripts; supports quick feedback during use.
INMP441 microphone module	Outside glove (near LCD)	I ² S (WS/SCK/SD)	Captures speech digitally via I ² S for cloud speech-to-text processing.
DFPlayer Mini (MP3 TF)	Inside glove (near controller) or beneath LCD	UART (serial mode)	Plays pre-recorded MP3 tracks from microSD/TF to produce audible phrases.
Speaker	Outside glove (near LCD)	Connected to DF-Player output	Emits the spoken output corresponding to recognized gestures/phrases.
LM2596 DC supply module	Inside glove (near battery/controller)	Power rail distribution	Regulates battery voltage to a stable DC output for the electronics.

4. Methods

The EchoHand system was implemented as a glove-mounted wearable platform. The physical layout was designed to meet three practical requirements: secure mechanical mounting during repeated hand motion, reliable electrical connections while the glove flexes, and acceptable comfort and visibility during use. As shown in Figure 1, the ESP32 microcontroller and its supporting electronics were assembled into a compact module and mounted inside the glove, typically near the wrist or the lower back-of-hand region.

This location keeps the palm and fingers unobstructed for natural signing and reduces the cable length between the finger sensors and the controller. The electronics were secured using Velcro straps and, when needed, adhesive fabric tape. Insulating materials such as

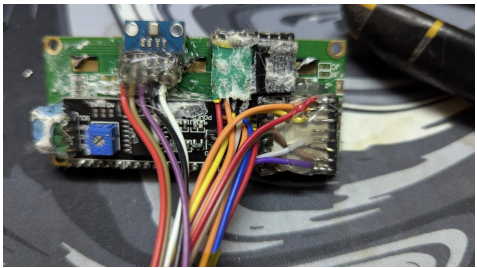


Figure 1. Controller module placement and overall assembly (image 33).

heat-shrink tubing, foam backing, or electrical tape were added to prevent short circuits and to avoid direct contact between conductive parts and the user's skin.

Finger posture sensing was achieved by installing one flex sensor on each finger along the dorsal surface. This placement concentrates bending at the finger joints and improves repeatability during motion. As illustrated in Figure 2, the sensors were taped to the fingers and their wires were routed toward the back of the glove. Each sensor was fixed at multiple points to prevent shifting, and the wire junction near the sensor tabs was strain-relieved by anchoring the wire to the glove immediately after the sensor end.



Figure 2. Sensor routing, LCD placement, and supporting modules (image 44).

This reduces mechanical stress on the sensor terminals and helps prevent fatigue failure during repeated flexion. Wiring was routed along glove seams where possible to minimize discomfort and reduce snagging. Electrically, each flex sensor functions as a variable resistor whose resistance increases with bending, enabling measurement through a voltage-divider configuration read by the ESP32 analog-to-digital converter (ADC).

User feedback was provided through a 16×2 LCD mounted on the back of the hand for direct visibility during signing, as shown in Figure 2. The inertial measurement unit (MPU6050) and the MP3 playback module were placed beneath or adjacent to the LCD region. This placement reduces wiring complexity and keeps the mass near the center of the hand, which improves wearability. The INMP441 microphone module and the speaker were also positioned near the LCD so that speech capture and audio output occur close to the main user interface. To support consistent speech acquisition, the microphone's acoustic port was kept unobstructed by fabric. The speaker was oriented to reduce blockage from glove material and to improve audibility.

Electrical integration followed a bus-oriented architecture aligned with the ESP32 peripheral interfaces. The LCD and the MPU6050 were connected using the I²C bus with shared SDA and SCL lines. The INMP441 was connected through I²S using dedicated WS, SCK, and SD signals configured in firmware, since the INMP441 outputs digital audio over I²S rather than an analog waveform. The DFPlayer Mini was controlled via UART in serial mode, allowing the ESP32 to trigger playback of pre-recorded MP3 tracks stored on a microSD or TF card. Flex sensors were read using ADC inputs on ADC1 pins. ADC1 was selected because ADC2 is not consistently available when Wi-Fi is active on the ESP32, so ADC1 reduces the risk of sampling conflicts during network operation.

Power was supplied through an LM2596 step-down (buck) regulator module that provides an adjustable regulated output suitable for embedded loads. In the glove assembly, the regulator and battery pack were typically placed near the controller to minimize wire length and reduce cable motion. All modules shared a common ground reference. Cable management, including short wire runs, secured connectors, and insulated solder joints, was treated as a core part of the integration method to reduce intermittent faults under movement.

On the firmware side, the ESP32 continuously sampled the flex sensors, normalized each channel using a user-assisted calibration routine that captured per-sensor minimum and maximum values, and applied a moving-average filter to reduce noise. Filtered readings were then discretized into three posture states (STRAIGHT, HALF_BENT, and BENT) and mapped to gestures using a dictionary-based recognition approach. In parallel, the

microphone stream was monitored using an energy-based voice activity detector (VAD). When speech-like activity exceeded a defined threshold, the device packaged the captured PCM audio into a WAV payload and transmitted it via HTTPS POST to Azure Speech-to-Text for transcription. This workflow aligns with the short-audio Speech-to-Text REST approach for synchronous transcription in cases where an application does not use the Speech SDK. The resulting transcript, along with gesture outputs, was displayed on the LCD. Selected gestures also triggered MP3 playback through the DFPlayer Mini.

5. Results

ECO_Hands was evaluated across ten trials for its two functional pipelines: (1) speech-to-text, in which spoken input is transcribed using the Azure Speech-to-Text service, and (2) sign-language-to-speech, in which glove-based gesture recognition produces a predefined phrase that is displayed and optionally played as audio. The evaluation was performed in both controlled use and field validation, including testing at the Egyptian Association for the Deaf and Mute, to reflect realistic interaction conditions such as variable signing styles, background noise, and natural differences in hand posture.

Performance was quantified using the word-level error metric shown in Equation (1). For each trial, the produced output phrase was compared against the intended phrase, and the number of incorrect words was divided by the total number of words in the phrase.

$$\text{Error Rate (\%)} = \frac{\text{Number of Incorrect Words}}{\text{Total Words in Phrase}} \times 100 \quad (1)$$

Across ten trials, the sign-language-to-speech pipeline achieved a mean error rate of 27.7%, corresponding to an average efficiency of 72.3% (where efficiency is interpreted as 100% – error rate). These results indicate that the gesture-driven pathway produced more reliable phrase output than the speech-driven pathway under the same evaluation approach.

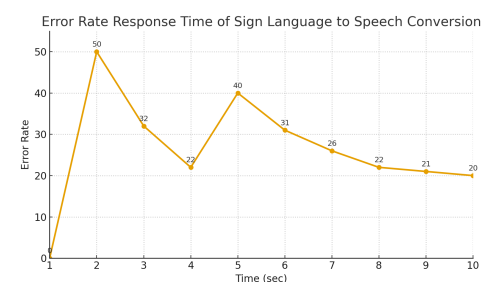


Figure 3. Sign language-to-speech error rate across ten trials.

Across ten trials, the speech-to-text pipeline achieved a mean error rate of 40.2%, corresponding to an average efficiency of 59.8% (where efficiency is interpreted as 100% – error rate). The higher mean error reflects sensitivity to acoustic conditions, speaking distance, background noise, and segmentation of the recorded window prior to cloud submission.

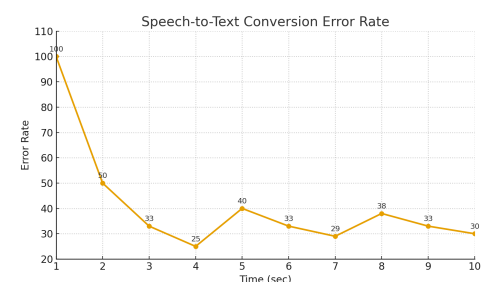


Figure 4. Speech-to-text error rate across ten trials.

The trends in Figures 3 and 4 show that both pipelines exhibit trial-to-trial variability. For sign-language-to-speech, error values decrease and stabilize in later trials, which is consistent with improved sensor placement stability and improved calibration consistency across repeated runs. For speech-to-text, early trials show higher error, followed by lower and more stable error values in later trials, which is consistent with tuning of voice activity detection sensitivity and improvements in microphone signal quality during system refinement. Importantly, the use of a word-level metric can yield large percentage changes for

short phrases; therefore, a single incorrect word may produce a high error rate for brief commands, and this effect should be considered when interpreting peaks.

5.1. Performance and Efficiency Improvements

The sign-language-to-speech pipeline outperformed speech-to-text primarily because it operates as a closed-vocabulary, dictionary-based classifier. Once finger bend signals are normalized and filtered, the system maps discrete finger-state patterns to a finite set of phrases, which reduces ambiguity and improves repeatability during field use. In contrast, speech-to-text performance depends on acoustic conditions, speaking distance, background noise, and accurate segmentation of the spoken utterance into the recorded window prior to cloud submission. In community settings such as the Egyptian Association for the Deaf and Mute, environmental noise and natural speaking variability can increase transcription error even when a high-quality cloud recognizer is used.

Efficiency improvements were achieved through iterative hardware and firmware refinement guided by observed failure modes. During early development, analog microphone modules produced high noise and inconsistent capture quality. The design was then migrated to a digital microphone approach, and the final selection of INMP441 provided cleaner and more consistent audio acquisition with the ESP32 I²S interface, improving the stability of speech capture prior to cloud submission. In addition, upgrading from a more memory-limited ESP32 configuration to ESP32 WROVER improved runtime robustness for buffering audio segments and operating multiple subsystems simultaneously, which reduced processing interruptions during recording and transmission.

Gesture performance improvements were driven by sensor reliability and signal conditioning. Unstable flex sensing was addressed by replacing erratic sensors and by using calibration-driven normalization so that each user's "straight" and "fully bent" postures define the effective measurement range. This reduces inter-user variability and improves state separability. The firmware further improved stability by applying a moving average filter to flex readings prior to discretization, reducing spurious state switching caused by micro-movements and electrical noise. For the speech pathway, efficiency improved through careful tuning of the voice activity threshold and recording window so that uploaded segments are more likely to contain complete words rather than truncated speech. Finally, multiple recognition backends were evaluated during development, and Azure was selected because it provided the most favorable balance of accuracy and responsiveness under the project's operating constraints, particularly during real-world testing sessions.

6. Discussion

6.1. System Overview

EchoHand is a wearable assistive system designed to turn a user's intent into clear outputs using two complementary inputs: hand gestures and speech. Gestures are detected through finger bend sensing, and speech is captured by a small digital microphone and transcribed using cloud speech recognition. As shown in Diagram 1, EchoHand splits computation between the ESP32 and Azure. The ESP32 handles gesture recognition locally to keep the device responsive and power-efficient, while Azure handles speech transcription, as cloud models typically achieve higher accuracy than compact on-device models.

In the firmware, the gesture pipeline runs continuously on the ESP32 (Code C4 to C7). Speech audio is uploaded only when voice activity is detected, so the system avoids constant streaming (Code C2, C8 to C9). This makes EchoHand a coordinated multimodal interface: the user can communicate through voice or gesture, and the firmware includes basic mechanisms intended to reduce conflicts between the two channels (Code C10).

6.2. Embedded Systems Integration

From an embedded systems viewpoint, the ESP32 WROVER is the integration core. It manages flex sensor sampling through the ADC (Code C4 to C6), communicates with the MPU6050 and the LCD over I2C (Code C3 and C10), captures audio from the INMP441 using I2S (Code C2), controls the DFPlayer Mini over UART (Code C10), and performs Wi-Fi networking plus HTTPS requests for cloud transcription (Code C1 and C9). It also hosts a local HTTP configuration page that allows parameter tuning, such as voice activity detection threshold and output volume, during testing (Code C1 and C10). In research terms, the firmware follows a standard embedded perception pipeline: raw signals are collected from sensors, processed into stable features, mapped into symbols such as gesture states or recognized text, and then presented through visual and audio outputs (Figure 1; Code C6 to C10).

6.3. Gesture Channel: Sensing, Calibration, and Recognition

A central part of the gesture channel is the use of flex sensors to represent finger posture in a compact and interpretable way. EchoHand reads five sensor channels (Index, Middle, Ring, Little, Thumb) using the ADC1 pins defined in the firmware (Code C4). However, to remain compatible with the current gesture vocabulary format, the recognizer intentionally uses only three fingers (Index, Middle, Thumb) for classification, even though all five sensors are still read and calibrated (Code C5 and C7). This keeps the early rule-based recognizer simpler and more stable, but it reduces how expressive the gesture set can be, especially for gestures that depend on ring or little finger posture.

Wearable sensing also varies across users and mounting conditions, so EchoHand includes an interactive calibration step at boot (Code C5). When calibration is enabled, the user records straight and fully bent reference points for each sensor, and each reference is computed as an average over 100 ADC readings (Code C5). These per-user reference values allow normalization by converting raw ADC readings into a 0 to 100% bend scale (Code C6). This improves portability because gesture definitions do not depend on absolute

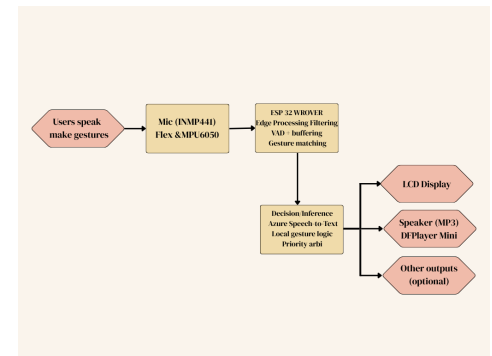


Figure 5. Diagram One

ADC numbers, and it also avoids assumptions about sensor direction because the firmware learns the effective minimum and maximum during calibration (Code C5 to C6). The tradeoff is that calibration quality matters: if the user does not fully straighten or fully bend during calibration, the usable range compresses and errors become more likely near decision thresholds.

After normalization, the firmware smooths each flex channel using a moving average filter with a window size of five samples (Code C6). This reduces noise from ADC quantization, wiring artifacts, and small hand movements, which helps prevent unstable outputs that rapidly switch states. The smoothed bend percentage is then discretized into three states, STRAIGHT, HALF_BENT, and BENT, using fixed thresholds at 33% and 66% (Code C6). This is academically attractive because it creates a transparent finite-state representation where each gesture is an explicit combination of finger states, which simplifies debugging and evaluation (Code C7). The limitation is that fixed thresholds can be sensitive to imperfect calibration and small posture differences, so scaling to larger gesture sets may benefit from adaptive thresholds or graded matching that tolerates near-boundary values.

Gesture recognition is implemented as a dictionary-driven matcher defined by a JSON configuration (Code C7). Each gesture entry maps an (Index, Middle, Thumb) state triple to a text label for the LCD and an audio track index for DFPlayer playback (Code C7 and C10). This approach is computationally inexpensive and easy to interpret because outputs are deterministic and directly traceable to specific dictionary entries. It also supports iterative development because new gestures can be added by editing the JSON rather than rewriting classifier logic. The main weakness is that matching currently requires exact equality across all three states (Code C7). In real use, small variations can push one finger across a threshold. A straightforward research extension is to add a closest-match strategy, such as similarity scoring over symbolic states or distance-based scoring over continuous bend percentages, so the system can select the most likely intended gesture under uncertainty.

6.4. Inertial Subsystem

The inertial subsystem, based on the MPU6050 accelerometer and gyroscope, provides a path to improve recognition beyond static finger posture. The firmware initializes the IMU, verifies connectivity, and runs a stationary calibration routine that estimates accelerometer and gyroscope offsets using repeated samples (Code C3). Bias estimation is important because offsets can distort motion features and contribute to drift. In the current implementation, the IMU values are calibrated but not yet integrated into the runtime gesture recognizer. This can be framed as staged development: the hardware and calibration workflow are validated (Code C3), and EchoHand is positioned for future integration of orientation-dependent or dynamic gesture features (Figure 1).

6.5. Speech Channel: Acquisition, VAD, and Cloud Transcription

The speech channel is built around the INMP441, an I2S digital output MEMS microphone. Audio is captured by the ESP32 as a digital stream configured at 16 kHz, 16-bit depth, and mono format (Code C2). Instead of sending audio continuously, EchoHand uses local gating so it uploads audio only when speech-like activity is detected. Voice activity detection is implemented using an energy-based method: the firmware processes one-second chunks, computes the mean absolute amplitude, and triggers speech processing when the average exceeds a threshold (Code C8). This threshold can be tuned through the local web interface, which helps adapt the system to different environments and microphone placements (Code C10). This design reduces unnecessary cloud calls and improves power and bandwidth efficiency. However, energy-based VAD can be sensitive to background

noise and speaking distance, and the current implementation relies primarily on this gating plus cloud robustness rather than explicit spectral noise suppression (Code C8).

When VAD triggers, the firmware records an additional fixed-duration segment to capture a more complete utterance, constructs a standard RIFF WAV header, and transmits the header plus PCM payload via an HTTPS POST request to the Azure speech recognition endpoint (Code C9). The JSON response is parsed and, when successful, the recognized transcript is displayed on the LCD with scrolling support for longer strings (Code C9 to C10). This pipeline matters academically because it demonstrates end-to-end integration across embedded acquisition, audio framing, cloud inference, and structured response parsing (Figure 1). It also enables higher-level behaviors because recognized text can be mapped to downstream actions, such as triggering specific outputs or guiding the user toward corresponding gestures (Code C9).

6.6. Outputs, Configuration, and Multimodal Arbitration

EchoHand provides immediate feedback through both visual and audio outputs. The LCDs short phrases and system states, including initialization messages, detection events, and speech transcripts, with scrolling for longer content (Code C10). In parallel, the DFPlayer Mini plays pre-recorded MP3 files associated with recognized gestures (Code C10). This pattern supports low-latency playback without requiring the ESP32 to decode MP3 audio in software. The local web server improves usability during testing by allowing quick volume adjustments and VAD threshold without reflashing firmware (Code C10).

A key challenge in multimodal interfaces is deciding which channel controls the output when both are active. Conceptually, EchoHand follows a speech-first policy: when speech is detected, speech output should take priority, and gestures should dominate only when speech is absent (Figure 1). In the current firmware, this priority is enforced mainly through sequential execution. Once VAD triggers, the device records audio and performs the HTTPS transaction, which temporarily blocks gesture updates and naturally biases behavior toward speech (Code C8 to C10). However, without an explicit arbitration state machine, gesture output can still compete with transcript display unless gesture actions are deliberately gated during the speech display window. The code uses timing flags such as `g_displayingAzure` and bounded display durations, but gesture actions are not fully suppressed during that interval (Code C10). A more rigorous approach is to implement an explicit output arbiter that locks the interface to speech output for a defined period while queuing or ignoring gesture outputs until the lock expires and to separate sensing, inference, and networking into distinct tasks to preserve responsiveness during cloud transactions (Code C10).

7. Acknowledgment

We sincerely thank Dr. Abdelrahman Elfikky for his continuous support and dedicated mentorship, particularly in supervising the project's code development and guiding technical enhancements that improved overall performance. We also express our gratitude to Dr. Ehab Elanany for his substantial assistance in testing and presenting the project, as well as for providing detailed, constructive feedback throughout the development process, including during the EISTF competition. Finally, we thank the Deputy of the Egyptian Association for the Deaf and Mute for facilitating access to real users and enabling practical evaluation of the system in an authentic setting.

8. EchoHand Glove Code (ECO_Hands Integrated Firmware)

Listing 1: –[C10] correspond to the code numbering used in the manuscript.]ECO_Hands integrated firmware (MPU6050 + flex calibration). The tags [C1]–[C10] correspond to the code numbering used in the manuscript.

```

1  /*
2  ECO_Hands (Integrated: MPU6050 + Flex Calibration)
3
4  Added:
5  - MPU6050 init + calibration routine (prints offsets on Serial)
6  - Interactive calibration for 5 flex sensors (min/max), updates g_flexMin/
7    g_flexMax at runtime
8  - Keeps your existing gesture JSON (3 states: Index, Middle, Thumb) by matching
9    ONLY those 3 sensors
10   while still reading/calibrating ALL 5 sensors.
11
12  How to use calibration:
13  - Open Serial Monitor (115200).
14  - On boot, press any key within 5 seconds to enter Calibration Mode.
15  - Follow prompts for MPU6050 + flex min/max.
16
17  -----
18  Code numbering (for paper references):
19  [C1] Wi-Fi / Azure configuration + endpoint + web server routing
20  [C2] INMP441 microphone (I2S) acquisition setup
21  [C3] MPU6050 initialization + calibration routine
22  [C4] Flex sensor hardware definition (5 channels)
23  [C5] Interactive calibration workflow (boot prompt + flex min/max capture)
24  [C6] Flex preprocessing: mapping + filtering + discretization
25  [C7] Gesture dictionary (JSON) + recognition + actions
26  [C8] Voice activity detection (VAD) gating
27  [C9] WAV framing + Azure STT request/response parsing + reverse mapping
28  [C10] Outputs/UI + main-loop coordination (LCD/DFPlayer/Web UI/timers)
29  -----
30  */
31
32  #include <WiFi.h>
33  #include <WiFiClientSecure.h>
34  #include <HTTPClient.h>
35  #include <ArduinoJson.h>
36  #include <driver/i2s.h>
37  #include <Wire.h>
38  #include "MPU6050.h"
39  #include <LiquidCrystal_I2C.h>
40  #include <DFRobotDFPlayerMini.h>
41  #include <HardwareSerial.h>
42  #include <WebServer.h>
43  #include <vector>
44
45  // ===== [C1] Wi-Fi / Azure Configuration =====
46
47  static String g_wifiSsid = "*****";
48  static String g_wifiPassword = "*****";
49
50  static const char* AZURE_API_KEY = "*****";
51  static const char* AZURE_REGION = "*****";

```

```

49 static const char* AZURE_LANG      = "en-US";
50
51 static String azureEndpoint() { // [C1]
52     return String("https://") + AZURE_REGION +
53         ".stt.speech.microsoft.com/speech/recognition/conversation/
54         cognitiveservices/v1?language=" +
55         AZURE_LANG;
56 }
57
58 // ===== [C2] INMP441 (I2S) Acquisition Constants
59 // =====
60 static constexpr int I2S_WS  = 14;
61 static constexpr int I2S_SD  = 15;
62 static constexpr int I2S_SCK = 25;
63
64 static constexpr uint32_t SAMPLE_RATE  = 16000;
65 static constexpr uint16_t SAMPLE_BITS  = 16;
66 static constexpr uint8_t  CHANNELS     = 1;
67
68 static constexpr uint16_t RECORD_SECONDS = 1; // additional seconds after VAD hit
69 static constexpr uint16_t CHUNK_SECONDS  = 1; // VAD window seconds
70
71 static constexpr size_t CHUNK_SAMPLES = (SAMPLE_RATE * CHANNELS * CHUNK_SECONDS);
72 static constexpr size_t CHUNK_BYTES  = (CHUNK_SAMPLES * (SAMPLE_BITS / 8));
73
74 static constexpr size_t REC_SAMPLES = (SAMPLE_RATE * CHANNELS * RECORD_SECONDS);
75 static constexpr size_t REC_BYTES  = (REC_SAMPLES * (SAMPLE_BITS / 8));
76
77 static int g_vadThreshold = 40;
78
79 static int16_t g_vadPcm[CHUNK_SAMPLES];
80 static uint8_t g_pcmPayload[CHUNK_BYTES + REC_BYTES];
81
82 // ===== [C3] MPU6050 (Init + Calibration)
83 // =====
84 static MPU6050 mpu;
85
86 struct MpuOffsets {
87     float ax = 0, ay = 0, az = 0;
88     float gx = 0, gy = 0, gz = 0;
89 };
90
91 static MpuOffsets g_mpuOffsets;
92 static constexpr int MPU_CALIB_SAMPLES = 1000;
93
94 // ===== [C4] Flex Sensors Hardware Definition (5)
95 // =====
96 // Pins chosen from your code; these are ADC1 pins (safe with WiFi on ESP32).
97 static constexpr int kFlexCount = 5;
98 static const int kFlexPins[kFlexCount] = {34, 35, 36, 39, 32};
99 static const char* kFingerNames[kFlexCount] = {"Index", "Middle", "Ring", "Little",
100 "Thumb"};
101
102 // Will be updated by calibration mode. Defaults here are placeholders.
103 static int g_flexMin[kFlexCount] = {431, 584, 467, 587, 815};

```

```

99 static int g_flexMax[kFlexCount] = {111, 259, 336, 300, 461};
100
101 // Thresholds in percent after mapping raw->0..100
102 static float g_flexThresholds[kFlexCount][2];
103
104 // ===== [C7] Gesture System Uses 3 Fingers
105 // =====
106 // Keep your existing gesture JSON format (3 states: Index, Middle, Thumb).
107 static constexpr int kGestureFingerCount = 3;
108 // Indices in the 5-sensor array: Index=0, Middle=1, Thumb=4
109 static const int kGestureFingerIdx[kGestureFingerCount] = {0, 1, 4};
110
111 // ===== [C6] Finger State Representation
112 // =====
113 enum class FingerState : uint8_t { STRAIGHT, HALF_BENT, BENT };
114
115 static const char* toString(FingerState s) {
116     switch (s) {
117         case FingerState::STRAIGHT: return "STRAIGHT";
118         case FingerState::HALF_BENT: return "HALF_BENT";
119         case FingerState::BENT:      return "BENT";
120     }
121     return "STRAIGHT";
122 }
123
124 static FingerState fromString(const String& sIn) {
125     String s = sIn;
126     s.toUpperCase();
127     if (s == "STRAIGHT") return FingerState::STRAIGHT;
128     if (s == "HALF_BENT") return FingerState::HALF_BENT;
129     if (s == "BENT")      return FingerState::BENT;
130     return FingerState::STRAIGHT;
131 }
132
133 static FingerState g_fingerStates[kFlexCount];
134
135 // ===== [C10] LCD + DFPlayer Output Subsystem
136 // =====
137 static LiquidCrystal_I2C lcd(0x27, 16, 2);
138 static HardwareSerial dfSerial(1);
139 static DFRobotDFPlayerMini mp3;
140 static int g_volumeValue = 30; // 0..30
141
142 // ===== [C10] Timers / Display Coordination
143 // =====
144 static constexpr uint32_t FLEX_INTERVAL_MS = 100;
145 static uint32_t g_prevFlexMs = 0;
146
147 static bool g_displayingAzure = false;
148 static uint32_t g_azureStartMs = 0;
149 static constexpr uint32_t AZURE_DISPLAY_MS = 5000;
150
151 static bool g_scrolling = false;
152 static String g_scrollText = "";
153 static uint32_t g_scrollPrevMs = 0;

```

```

150 static constexpr uint32_t SCROLL_INTERVAL_MS = 150;
151 static int g_scrollPos = 0;
152
153 static bool g_reverseDisplayed = false;
154 static uint32_t g_reverseStartMs = 0;
155 static constexpr uint32_t REVERSE_DISPLAY_MS = 3000;
156
157 // ===== [C6] Moving Average Filter (Flex Smoothing)
158 // =====
159 template <size_t Window>
160 class MovingAverage {
161 public:
162     void reset() {
163         idx_ = 0; count_ = 0; sum_ = 0;
164         for (size_t i = 0; i < Window; ++i) buf_[i] = 0;
165     }
166
167     int add(int sample) {
168         if (count_ < Window) {
169             buf_[idx_] = sample;
170             sum_ += sample;
171             ++count_;
172             idx_ = (idx_ + 1) % Window;
173             return (int)(sum_ / (long)count_);
174         }
175         sum_ -= buf_[idx_];
176         buf_[idx_] = sample;
177         sum_ += sample;
178         idx_ = (idx_ + 1) % Window;
179         return (int)(sum_ / (long)Window);
180     }
181 private:
182     int buf_[Window];
183     size_t idx_ = 0, count_ = 0;
184     long sum_ = 0;
185 };
186
187 static constexpr size_t FILTER_SIZE = 5;
188 static MovingAverage<FILTER_SIZE> g_flexFilters[kFlexCount];
189
190 // ===== [C7] Gesture Config JSON (3-finger)
191 // =====
192 static const char* gestureConfigJson = R"RAW(
193 {
194     "gestures": [
195         { "name": "Hello", "fingerStates": ["BENT","BENT","STRAIGHT"], "mp3Track": 2, "
196           displayText": "Hello" },
197         { "name": "Thank you", "fingerStates": ["STRAIGHT","BENT","STRAIGHT"], "
198           mp3Track": 3, "displayText": "Thank You" },
199         { "name": "Sorry", "fingerStates": ["BENT","BENT","BENT"], "mp3Track": 4, "
200           displayText": "Sorry" },
201         { "name": "Please", "fingerStates": ["STRAIGHT","BENT","BENT"], "mp3Track": 5,
202           "displayText": "Please" },

```



```

198     { "name": "Help", "fingerStates": ["BENT","STRAIGHT","BENT"], "mp3Track": 6, "
      displayText": "Help" },
199     { "name": "I need water", "fingerStates": ["BENT","HALF_BENT","BENT"], "
      mp3Track": 7, "displayText": "I need water" },
200     { "name": "I'm hungry", "fingerStates": ["STRAIGHT","STRAIGHT","BENT"], "
      mp3Track": 8, "displayText": "I'm hungry" },
201     { "name": "I want to go to the bath", "fingerStates": ["BENT","STRAIGHT","
      HALF_BENT"], "mp3Track": 9, "displayText": "I want to go to the bath" },
202     { "name": "Yes", "fingerStates": ["STRAIGHT","STRAIGHT","STRAIGHT"], "mp3Track"
      : 10, "displayText": "Yes" },
203     { "name": "No", "fingerStates": ["BENT","STRAIGHT","STRAIGHT"], "mp3Track": 11,
      "displayText": "No" },
204     { "name": "Stop", "fingerStates": ["STRAIGHT","HALF_BENT","STRAIGHT"], "
      mp3Track": 12, "displayText": "Stop" },
205     { "name": "Where is", "fingerStates": ["STRAIGHT","BENT","HALF_BENT"], "
      mp3Track": 13, "displayText": "Where is" },
206     { "name": "Call police", "fingerStates": ["STRAIGHT","HALF_BENT","HALF_BENT"],
      "mp3Track": 14, "displayText": "Call police" },
207     { "name": "Call ambulance", "fingerStates": ["BENT","HALF_BENT","STRAIGHT"], "
      mp3Track": 15, "displayText": "Call ambulance" },
208     { "name": "I want to cross the street", "fingerStates": ["HALF_BENT","BENT","
      STRAIGHT"], "mp3Track": 16, "displayText": "I want to cross the street" }
209 ]
210 }
211 )RAW";
212
213 struct GestureDefinition {
214     String name;
215     FingerState required[kGestureFingerCount];
216     int mp3Track = 0;
217     String displayText;
218 };
219
220 static std::vector<GestureDefinition> g_gestures;
221 static int g_lastGestureIndex = -1;
222
223 // ===== [C9] Reverse Dictionary (Speech-to-Gesture Hint)
224 // =====
225 struct ReverseMapping {
226     const char* phrase;
227     FingerState states[kGestureFingerCount];
228 };
229
230 static const ReverseMapping g_reverseDict[] = {
231     {"HELLO", {FingerState::BENT, FingerState::BENT, FingerState::STRAIGHT}},
232     {"BYE", {FingerState::STRAIGHT, FingerState::STRAIGHT, FingerState::BENT}}
233 };
234
235 static constexpr size_t g_reverseDictSize = sizeof(g_reverseDict) / sizeof(
236     g_reverseDict[0]);
237
238 // ===== [C10] Web Server =====
239 static WebServer server(80);
240
241 // ===== [C10] LCD Helpers =====

```

```

240 static void lcdPrintLine(uint8_t row, const String& text) {
241     lcd.setCursor(0, row);
242     String t = text;
243     if (t.length() > 16) t = t.substring(0, 16);
244     lcd.print(t);
245     for (int i = (int)t.length(); i < 16; ++i) lcd.print(' ');
246 }
247
248 static void startScroll(const String& text) {
249     g_scrollText = text;
250     g_scrollPos = 0;
251     g_scrolling = true;
252     g_scrollPrevMs = millis();
253 }
254
255 static void updateScroll() {
256     if (!g_scrolling) return;
257     uint32_t now = millis();
258     if (now - g_scrollPrevMs < SCROLL_INTERVAL_MS) return;
259     g_scrollPrevMs = now;
260
261     if (g_scrollPos >= (int)g_scrollText.length()) {
262         g_scrolling = false;
263         return;
264     }
265
266     int endPos = g_scrollPos + 16;
267     if (endPos > (int)g_scrollText.length()) endPos = (int)g_scrollText.length();
268     String slice = g_scrollText.substring(g_scrollPos, endPos);
269     while (slice.length() < 16) slice += " ";
270
271     lcdPrintLine(1, slice);
272     g_scrollPos += 2;
273 }
274
275 // ===== [C5] Calibration Mode (Boot Prompt + Flex Min/Max)
276 // =====
277 static bool promptCalibration(uint32_t timeoutMs) { // [C5]
278     Serial.println();
279     Serial.println("Press any key within 5 seconds to enter CALIBRATION MODE...");
280     lcdPrintLine(0, "Press key:Calib");
281     lcdPrintLine(1, "5 seconds...");
282
283     uint32_t start = millis();
284     while (millis() - start < timeoutMs) {
285         if (Serial.available() > 0) {
286             while (Serial.available()) Serial.read();
287             Serial.println("Entering calibration mode.");
288             lcdPrintLine(0, "Calibration");
289             lcdPrintLine(1, "Starting...");
290             return true;
291         }
292         delay(20);
293     }
294     Serial.println("Skipping calibration mode.");

```

```

294     return false;
295 }
296
297 // ===== [C3] MPU6050 Calibration Routine
298 // =====
299 static void calibrateMPU6050() { // [C3]
300     Serial.println();
301     Serial.print("Calibrating MPU6050 with ");
302     Serial.print(MPU_CALIB_SAMPLES);
303     Serial.println(" samples...");
304     lcdPrintLine(0, "MPU6050 Calib");
305     lcdPrintLine(1, "Hold still...");
306
307     int16_t ax, ay, az, gx, gy, gz;
308     int64_t ax_sum = 0, ay_sum = 0, az_sum = 0;
309     int64_t gx_sum = 0, gy_sum = 0, gz_sum = 0;
310
311     for (int i = 0; i < MPU_CALIB_SAMPLES; i++) {
312         mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
313         ax_sum += ax; ay_sum += ay; az_sum += az;
314         gx_sum += gx; gy_sum += gy; gz_sum += gz;
315         delay(2);
316     }
317
318     g_mpuOffsets.ax = (float)ax_sum / MPU_CALIB_SAMPLES;
319     g_mpuOffsets.ay = (float)ay_sum / MPU_CALIB_SAMPLES;
320     g_mpuOffsets.az = ((float)az_sum / MPU_CALIB_SAMPLES) - 16384.0f; // assumes +/-2
321     g
322
323     g_mpuOffsets.gx = (float)gx_sum / MPU_CALIB_SAMPLES;
324     g_mpuOffsets.gy = (float)gy_sum / MPU_CALIB_SAMPLES;
325     g_mpuOffsets.gz = (float)gz_sum / MPU_CALIB_SAMPLES;
326
327     Serial.println("MPU6050 calibration complete. Offsets:");
328     Serial.println("Accelerometer:");
329     Serial.print("  ax_offset: "); Serial.println(g_mpuOffsets.ax);
330     Serial.print("  ay_offset: "); Serial.println(g_mpuOffsets.ay);
331     Serial.print("  az_offset: "); Serial.println(g_mpuOffsets.az);
332     Serial.println("Gyroscope:");
333     Serial.print("  gx_offset: "); Serial.println(g_mpuOffsets.gx);
334     Serial.print("  gy_offset: "); Serial.println(g_mpuOffsets.gy);
335     Serial.print("  gz_offset: "); Serial.println(g_mpuOffsets.gz);
336
337     lcdPrintLine(0, "MPU Calib Done");
338     lcdPrintLine(1, "See Serial");
339     delay(1500);
340 }
341
342 // ===== [C5] Flex Calibration Helpers
343 // =====
344 static void waitForUserInput() {
345     while (Serial.available() == 0) { delay(10); }
346     while (Serial.available()) Serial.read();
347 }

```

```

346 static int readAverageAnalog(int pin) { // [C5]
347     const int numReadings = 100;
348     int64_t sum = 0;
349     for (int i = 0; i < numReadings; i++) {
350         sum += analogRead(pin);
351         delay(1);
352     }
353     return (int)(sum / numReadings);
354 }
355
356 static void calibrateFlexSensors() { // [C5]
357     Serial.println();
358     Serial.println("Flex Sensors Calibration");
359     Serial.println("=====");
360
361     // Straight (min)
362     Serial.println("1) Keep all fingers STRAIGHT.");
363     Serial.println("Press any key to record MIN values...");
364     lcdPrintLine(0, "Flex Calib MIN");
365     lcdPrintLine(1, "Straight+Key");
366     waitForUserInput();
367
368     Serial.println("Recording MIN (straight) values...");
369     for (int i = 0; i < kFlexCount; i++) {
370         g_flexMin[i] = readAverageAnalog(kFlexPins[i]);
371         Serial.print(kFingerNames[i]); Serial.print(" Min: ");
372         Serial.println(g_flexMin[i]);
373     }
374
375     // Bent (max)
376     Serial.println();
377     Serial.println("2) Fully BEND all fingers.");
378     Serial.println("Press any key to record MAX values...");
379     lcdPrintLine(0, "Flex Calib MAX");
380     lcdPrintLine(1, "Bent+Key");
381     waitForUserInput();
382
383     Serial.println("Recording MAX (bent) values...");
384     for (int i = 0; i < kFlexCount; i++) {
385         g_flexMax[i] = readAverageAnalog(kFlexPins[i]);
386         Serial.print(kFingerNames[i]); Serial.print(" Max: ");
387         Serial.println(g_flexMax[i]);
388     }
389
390     Serial.println();
391     Serial.println("Calibration Complete. Use these values:");
392     Serial.println("flexMin = {");
393     for (int i = 0; i < kFlexCount; i++) {
394         Serial.print(" "); Serial.print(g_flexMin[i]);
395         Serial.println(i == kFlexCount - 1 ? "" : ",");
396     }
397     Serial.println("}");
398     Serial.println("flexMax = {");
399     for (int i = 0; i < kFlexCount; i++) {
400         Serial.print(" "); Serial.print(g_flexMax[i]);

```

```

401     Serial.println(i == kFlexCount - 1 ? "" : ",");
402 }
403 Serial.println("};");
404
405 lcdPrintLine(0, "Flex Calib Done");
406 lcdPrintLine(1, "See Serial");
407 delay(1500);
408 }
409
410 // ===== [C1] WiFi Setup =====
411 static void setupWiFi() { // [C1]
412     Serial.println("Connecting to Wi-Fi...");
413     WiFi.mode(WIFI_STA);
414     WiFi.begin(g_wifiSsid.c_str(), g_wifiPassword.c_str());
415     while (WiFi.status() != WL_CONNECTED) {
416         delay(500);
417         Serial.print(".");
418     }
419     Serial.print("\nConnected. IP: ");
420     Serial.println(WiFi.localIP());
421 }
422
423 // ===== [C2] Microphone / I2S Setup
424 // =====
425 static void setupMic() { // [C2]
426     i2s_config_t i2sConfig = {};
427     i2sConfig.mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX);
428     i2sConfig.sample_rate = SAMPLE_RATE;
429     i2sConfig.bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT;
430     i2sConfig.channel_format = I2S_CHANNEL_FMT_ONLY_LEFT;
431     i2sConfig.communication_format = I2S_COMM_FORMAT_I2S;
432     i2sConfig.intr_alloc_flags = ESP_INTR_FLAG_LEVEL1;
433     i2sConfig.dma_buf_count = 8;
434     i2sConfig.dma_buf_len = 64;
435     i2sConfig.use_apll = false;
436     i2sConfig.tx_desc_auto_clear = false;
437     i2sConfig.fixed_mclk = 0;
438
439     i2s_pin_config_t pinConfig = {};
440     pinConfig.bck_io_num = I2S_SCK;
441     pinConfig.ws_io_num = I2S_WS;
442     pinConfig.data_out_num = I2S_PIN_NO_CHANGE;
443     pinConfig.data_in_num = I2S_SD;
444
445     esp_err_t err = i2s_driver_install(I2S_NUM_0, &i2sConfig, 0, nullptr);
446     if (err != ESP_OK) {
447         Serial.printf("I2S install failed: %d\n", err);
448         while (true) delay(1000);
449     }
450
451     err = i2s_set_pin(I2S_NUM_0, &pinConfig);
452     if (err != ESP_OK) {
453         Serial.printf("I2S pin set failed: %d\n", err);
454         while (true) delay(1000);
455     }

```

```

455     i2s_zero_dma_buffer(I2S_NUM_0);
456     Serial.println("Microphone initialized.");
457 }
458
459 // ===== [C6] Flex Logic (Thresholds + Discretization)
460 // =====
461 static void calculateFlexThresholds() { // [C6]
462     for (int i = 0; i < kFlexCount; i++) {
463         g_flexThresholds[i][0] = 33.0f;
464         g_flexThresholds[i][1] = 66.0f;
465     }
466 }
467
468 static void classifyFingerStates(const int flexPercent[kFlexCount]) { // [C6]
469     for (int i = 0; i < kFlexCount; i++) {
470         const float th0 = g_flexThresholds[i][0];
471         const float th1 = g_flexThresholds[i][1];
472         const int v = flexPercent[i];
473
474         if (v < th0) g_fingerStates[i] = FingerState::STRAIGHT;
475         else if (v < th1) g_fingerStates[i] = FingerState::HALF_BENT;
476         else g_fingerStates[i] = FingerState::BENT;
477     }
478 }
479
480 // ===== [C7] Gesture Parsing / Recognition
481 // =====
482 static void parseGestureConfig(const char* json) { // [C7]
483     StaticJsonDocument<8192> doc;
484     DeserializationError error = deserializeJson(doc, json);
485     if (error) {
486         Serial.print("Gesture JSON parse error: ");
487         Serial.println(error.c_str());
488         return;
489     }
490
491     JsonArray arr = doc["gestures"].as<JsonArray>();
492     if (arr.isNull()) {
493         Serial.println("Gesture JSON missing 'gestures' array.");
494         return;
495     }
496
497     g_gestures.clear();
498     g_gestures.reserve(arr.size());
499
500     for (JsonObject obj : arr) {
501         GestureDefinition def;
502         def.name = (const char*)(obj["name"] | "");
503         def.mp3Track = (int)(obj["mp3Track"] | 0);
504         def.displayText = (const char*)(obj["displayText"] | "");
505
506         JsonArray fs = obj["fingerStates"].as<JsonArray>();
507         if ((int)fs.size() != kGestureFingerCount) {
508             Serial.println("Gesture skipped: fingerStates size mismatch.");
509         }
510     }
511 }

```



```

508         continue;
509     }
510
511     for (int i = 0; i < kGestureFingerCount; i++) {
512         def.required[i] = fromString(fs[i].as<String>());
513     }
514
515     g_gestures.push_back(def);
516 }
517
518 Serial.printf("Loaded %d gestures.\n", (int)g_gestures.size());
519 }
520
521 static bool gestureMatches(const GestureDefinition& g) { // [C7]
522     for (int i = 0; i < kGestureFingerCount; i++) {
523         const int sensorIdx = kGestureFingerIdx[i];
524         if (g_fingerStates[sensorIdx] != g.required[i]) return false;
525     }
526     return true;
527 }
528
529 static void performGestureAction(int idx) { // [C7]
530     GestureDefinition& g = g_gestures[idx];
531     Serial.println("Gesture recognized: " + g.name);
532
533     lcdPrintLine(0, g.name);
534
535     if (g.mp3Track > 0) mp3.play(g.mp3Track);
536
537     if (g.displayText.length() <= 16) {
538         lcdPrintLine(1, g.displayText);
539         g_scrolling = false;
540     } else {
541         startScroll(g.displayText);
542     }
543 }
544
545 static void recognizeGestureFromJSON() { // [C7]
546     for (int i = 0; i < (int)g_gestures.size(); i++) {
547         if (gestureMatches(g_gestures[i])) {
548             if (g_lastGestureIndex != i) {
549                 performGestureAction(i);
550                 g_lastGestureIndex = i;
551             }
552             return;
553         }
554     }
555
556     g_lastGestureIndex = -1;
557     if (!g_reverseDisplayed) lcdPrintLine(0, "No Gesture");
558 }
559
560 // ===== [C9] WAV Framing + Azure STT
561 // =====
562 static void addWavHeader(uint8_t* wav, uint32_t pcmBytes) { // [C9]

```

```

562     memcpy(wav + 0, "RIFF", 4);
563     uint32_t fileSize = pcmBytes + 36;
564     memcpy(wav + 4, &fileSize, 4);
565     memcpy(wav + 8, "WAVE", 4);
566
567     memcpy(wav + 12, "fmt ", 4);
568     uint32_t fmtChunkSize = 16;
569     memcpy(wav + 16, &fmtChunkSize, 4);
570
571     uint16_t audioFormat = 1;
572     memcpy(wav + 20, &audioFormat, 2);
573
574     uint16_t numChannels = CHANNELS;
575     memcpy(wav + 22, &numChannels, 2);
576
577     uint32_t sampleRate = SAMPLE_RATE;
578     memcpy(wav + 24, &sampleRate, 4);
579
580     uint32_t byteRate = SAMPLE_RATE * CHANNELS * (SAMPLE_BITS / 8);
581     memcpy(wav + 28, &byteRate, 4);
582
583     uint16_t blockAlign = CHANNELS * (SAMPLE_BITS / 8);
584     memcpy(wav + 32, &blockAlign, 2);
585
586     uint16_t bitsPerSample = SAMPLE_BITS;
587     memcpy(wav + 34, &bitsPerSample, 2);
588
589     memcpy(wav + 36, "data", 4);
590     memcpy(wav + 40, &pcmBytes, 4);
591 }
592
593 class AudioStream : public Stream { // [C9]
594 public:
595     AudioStream(const uint8_t* header, size_t headerSize, const uint8_t* pcm, size_t
        pcmSize)
596         : header_(header), headerSize_(headerSize), pcm_(pcm), pcmSize_(pcmSize) {}
597
598     int read() override {
599         if (hPos_ < headerSize_) return header_[hPos_++];
600         if (pPos_ < pcmSize_) return pcm_[pPos_++];
601         return -1;
602     }
603
604     int available() override {
605         if (hPos_ < headerSize_) return (int)(headerSize_ - hPos_);
606         if (pPos_ < pcmSize_) return (int)(pcmSize_ - pPos_);
607         return 0;
608     }
609
610     int peek() override {
611         if (hPos_ < headerSize_) return header_[hPos_];
612         if (pPos_ < pcmSize_) return pcm_[pPos_];
613         return -1;
614     }
615

```

```

616 void flush() override {}
617 size_t write(uint8_t) override { return 0; }
618
619 private:
620     const uint8_t* header_;
621     size_t headerSize_;
622     size_t hPos_ = 0;
623
624     const uint8_t* pcm_;
625     size_t pcmSize_;
626     size_t pPos_ = 0;
627 };
628
629 // ===== [C8] Voice Activity Detection (VAD)
630 // =====
631 static bool detectVoice(const int16_t* pcm, size_t samples) { // [C8]
632     int64_t sum = 0;
633     for (size_t i = 0; i < samples; i++) {
634         int32_t v = pcm[i];
635         if (v < 0) v = -v;
636         sum += v;
637     }
638     int32_t avg = (samples > 0) ? (int32_t)(sum / (int64_t)samples) : 0;
639     Serial.printf("VAD avg=%ld (th=%d)\n", (long)avg, g_vadThreshold);
640     return avg > g_vadThreshold;
641 }
642
643 // ===== [C9] Reverse Mapping + Azure Response Parsing
644 // =====
645 static void showReverseGesture(const String& phraseUpper) { // [C9]
646     for (size_t i = 0; i < g_reverseDictSize; i++) {
647         if (phraseUpper.indexOf(g_reverseDict[i].phrase) >= 0) {
648             Serial.println("Reverse gesture for: " + String(g_reverseDict[i].phrase));
649
650             lcdPrintLine(0, "Rev: " + String(g_reverseDict[i].phrase));
651             lcdPrintLine(1, "");
652
653             // Debug for the 3 gesture fingers
654             for (int f = 0; f < kGestureFingerCount; f++) {
655                 const int idx = kGestureFingerIdx[f];
656                 Serial.printf("%s => %s | ", kFingerNames[idx], toString(g_reverseDict[i].
657 states[f]));
658             }
659             Serial.println();
660
661             g_reverseDisplayed = true;
662             g_reverseStartMs = millis();
663             return;
664         }
665     }
666 }
667
668 static void parseAzureResponse(const String& response) { // [C9]
669     StaticJsonDocument<4096> doc;
670     DeserializationError error = deserializeJson(doc, response);

```

```

668     if (error) {
669         Serial.printf("Azure JSON parse failed: %s\n", error.c_str());
670         lcdPrintLine(1, "JSON Parse Err");
671         return;
672     }
673
674     const char* status = doc["RecognitionStatus"];
675     if (!status) {
676         Serial.println("Azure response missing RecognitionStatus.");
677         return;
678     }
679
680     if (strcmp(status, "Success") == 0) {
681         const char* text = doc["DisplayText"] | "";
682         Serial.printf("Transcribed: %s\n", text);
683
684         lcdPrintLine(0, "Azure:");
685         if (strlen(text) <= 16) {
686             lcdPrintLine(1, text);
687             g_scrolling = false;
688         } else {
689             startScroll(String(text));
690         }
691
692         String phrase = String(text);
693         phrase.toUpperCase();
694         showReverseGesture(phrase);
695
696     } else {
697         Serial.printf("Azure status: %s\n", status);
698         lcdPrintLine(0, "Azure Status:");
699         lcdPrintLine(1, status);
700     }
701
702     g_displayingAzure = true;
703     g_azureStartMs = millis();
704 }
705
706 static void recordAndSendAudio(const uint8_t* initialPcmBytes, size_t initialLen) {
707     // [C9]
708     size_t additionalLen = 0;
709
710     size_t bytesRead = 0;
711     i2s_read(I2S_NUM_0, g_pcmPayload + initialLen, REC_BYTES, &bytesRead,
712             portMAX_DELAY);
713     additionalLen = bytesRead;
714     Serial.printf("Additional recording bytes: %d\n", (int)additionalLen);
715
716     const size_t totalPcmLen = initialLen + additionalLen;
717     if (totalPcmLen > sizeof(g_pcmPayload)) {
718         Serial.println("PCM buffer overflow (unexpected).");
719         lcdPrintLine(1, "PCM Overflow");
720         return;
721     }

```

```

721     uint8_t wavHeader[44];
722     addWavHeader(wavHeader, (uint32_t)totalPcmLen);
723
724     AudioStream stream(wavHeader, sizeof(wavHeader), g_pcmPayload, totalPcmLen);
725     const size_t payloadSize = sizeof(wavHeader) + totalPcmLen;
726
727     WiFiClientSecure client;
728     client.setInsecure();
729
730     HTTPClient https;
731     const String url = azureEndpoint();
732
733     if (!https.begin(client, url)) {
734         Serial.println("Azure connect failed.");
735         lcdPrintLine(1, "Azure Conn Err");
736         return;
737     }
738
739     https.addHeader("Content-Type", "audio/wav");
740     https.addHeader("Ocp-Apim-Subscription-Key", AZURE_API_KEY);
741     https.addHeader("Content-Length", String(payloadSize));
742
743     int code = https.sendRequest("POST", &stream, payloadSize);
744     if (code > 0) {
745         Serial.printf("Azure HTTP code: %d\n", code);
746         String resp = https.getString();
747         parseAzureResponse(resp);
748     } else {
749         Serial.printf("Azure request error: %s\n", https.errorToString(code).c_str());
750         lcdPrintLine(1, "Azure Req Err");
751     }
752
753     https.end();
754 }
755
756 // ===== [C10] Web Pages (Settings + Gestures UI)
757 // =====
758 static String htmlHeader(const String& title) {
759     String s;
760     s.reserve(600);
761     s += "<!DOCTYPE html><html><head><meta charset='UTF-8' />";
762     s += "<meta name='viewport' content='width=device-width, initial-scale=1' />";
763     s += "<title>" + title + "</title>";
764     s += "<style>"
765         "body{font-family:Arial,sans-serif;margin:20px;background:#f3f3f3;}"
766         ".container{max-width:520px;margin:auto;background:#fff;padding:20px;border-
767         radius:8px;}"
768         "h2{text-align:center;}"
769         "label{display:block;margin-top:10px;}"
770         "input[type=text],input[type=password]{width:100%;padding:8px;margin-top:5px
771         ;box-sizing:border-box;}"
772         ".slider-value{font-weight:bold;margin-left:10px;}"
773         ".slider-container{display:flex;align-items:center;gap:10px;}"
774         ".btn{margin-top:20px;background:#007BFF;color:#fff;border:none;padding:10px
775         ;width:100%;cursor:pointer;";

```

```

772         "border-radius:5px;font-size:16px;}"
773         ".btn:hover{background:#0056b3;}"
774         ".nav{margin-bottom:20px;text-align:center;}"
775         ".nav a{margin:0 10px;text-decoration:none;color:#007BFF;}"
776         "table{width:100%;border-collapse:collapse;}"
777         "th,td{border:1px solid #ddd;padding:8px;text-align:center;}"
778         "th{background:#007BFF;color:#fff;}"
779         "</style></head><body>";
780     return s;
781 }
782
783 static void handleRootPage() {
784     String page = htmlHeader("Settings");
785     page += "<div class='container'>"
786             "<div class='nav'>"
787             "<a href='/settings'>Settings</a> | "
788             "<a href='/gestures'>Gestures</a>"
789             "</div>"
790             "<h2>Network & Audio Settings</h2>"
791             "<form action='/submit' method='POST'>"
792             "<label for='ssid'>WiFi SSID:</label>"
793             "<input type='text' id='ssid' name='ssid' placeholder='Enter WiFi SSID'"
794             "required>"
795             "<label for='password'>WiFi Password:</label>"
796             "<input type='password' id='password' name='password' placeholder='Enter"
797             "WiFi Password'>"
798             "<label for='volume'>DFPlayer Volume:</label>"
799             "<div class='slider-container'>"
800             "<input type='range' id='volume' name='volume' min='0' max='30' value='"
801             + String(g_volumeValue) +
802             "' oninput='volumeVal.innerText=this.value'>"
803             "<span class='slider-value' id='volumeVal'>" + String(g_volumeValue) + "
804             "</span>"
805             "</div>"
806             "<label for='vad'>VAD Threshold:</label>"
807             "<div class='slider-container'>"
808             "<input type='range' id='vad' name='vad' min='10' max='100' value='" +
809             String(g_vadThreshold) +
810             "' oninput='vadVal.innerText=this.value'>"
811             "<span class='slider-value' id='vadVal'>" + String(g_vadThreshold) + "</"
812             "span>"
813             "</div>"
814             "<button type='submit' class='btn'>Save</button>"
815             "</form></div></body></html>";
816     server.send(200, "text/html", page);
817 }
818
819 static void handleGesturesPage() {
820     String page = htmlHeader("Gestures");
821     page += "<div class='container'>"

```

```

821         "<div class='nav'>"
822         "<a href='/settings'>Settings</a> | "
823         "<a href='/gestures'>Gestures</a>"
824         "</div>"
825         "<h2>Available Gestures (Index, Middle, Thumb)</h2>"
826         "<table><tr><th>Name</th><th>Finger States</th><th>MP3 Track</th><th>
Display Text</th></tr>";
827
828     for (int i = 0; i < (int)g_gestures.size(); i++) {
829         String statesStr;
830         for (int f = 0; f < kGestureFingerCount; f++) {
831             statesStr += toString(g_gestures[i].required[f]);
832             if (f < kGestureFingerCount - 1) statesStr += ", ";
833         }
834
835         page += "<tr><td>" + g_gestures[i].name + "</td>"
836             "<td>" + statesStr + "</td>"
837             "<td>" + String(g_gestures[i].mp3Track) + "</td>"
838             "<td>" + g_gestures[i].displayText + "</td></tr>";
839     }
840
841     page += "</table></div></body></html>";
842     server.send(200, "text/html", page);
843 }
844
845 static void handleFormSubmit() {
846     if (server.method() != HTTP_POST) {
847         server.send(405, "text/plain", "Method Not Allowed");
848         return;
849     }
850
851     if (server.hasArg("ssid")) g_wifiSsid = server.arg("ssid");
852     if (server.hasArg("password")) g_wifiPassword = server.arg("password");
853
854     if (server.hasArg("volume")) {
855         g_volumeValue = server.arg("volume").toInt();
856         g_volumeValue = constrain(g_volumeValue, 0, 30);
857         mp3.volume(g_volumeValue);
858     }
859
860     if (server.hasArg("vad")) {
861         g_vadThreshold = server.arg("vad").toInt();
862         g_vadThreshold = constrain(g_vadThreshold, 10, 100);
863     }
864
865     String page = htmlHeader("Updated");
866     page += "<div class='container'>"
867         "<h2>Settings Updated</h2>"
868         "<p><b>SSID:</b> " + g_wifiSsid + "<br>"
869         "<b>Password:</b> (hidden)<br>"
870         "<b>Volume:</b> " + String(g_volumeValue) + "<br>"
871         "<b>VAD Threshold:</b> " + String(g_vadThreshold) + "</p>"
872         "<p><a href='/settings'>Back</a></p>"
873         "</div></body></html>";
874

```



```

875     server.send(200, "text/html", page);
876 }
877
878 // ===== [C10] Arduino Setup / Loop (Coordination) =====
879 void setup() {
880     Serial.begin(115200);
881     while (!Serial) {}
882
883     Wire.begin();
884
885     lcd.init();
886     lcd.backlight();
887     lcdPrintLine(0, "Initializing...");
888     lcdPrintLine(1, "Please wait");
889     delay(300);
890
891     // [C3] MPU init + wiring validation
892     Serial.println("Initializing MPU6050...");
893     mpu.initialize();
894     if (!mpu.testConnection()) {
895         Serial.println("MPU6050 connection failed!");
896         lcdPrintLine(0, "MPU6050 Error");
897         lcdPrintLine(1, "Check wiring");
898         while (true) delay(1000);
899     }
900     Serial.println("MPU6050 connected successfully.");
901
902     // [C5] Optional interactive calibration at boot
903     const bool doCalib = promptCalibration(5000);
904     if (doCalib) {
905         Serial.println("Place MPU6050 on a flat stable surface.");
906         Serial.println("Starting MPU calibration in 2 seconds...");
907         lcdPrintLine(0, "Place flat");
908         lcdPrintLine(1, "MPU in 2s");
909         delay(2000);
910
911         calibrateMPU6050(); // [C3]
912         calibrateFlexSensors(); // [C5]
913     }
914
915     // [C1] Wi-Fi init
916     setupWiFi();
917
918     // [C10] Web server routes
919     server.on("/", handleRootPage);
920     server.on("/settings", handleRootPage);
921     server.on("/gestures", handleGesturesPage);
922     server.on("/submit", handleFormSubmit);
923     server.begin();
924     Serial.println("Web server started.");
925
926     // [C2] Mic init
927     setupMic();
928

```

```

929 // [C10] DFPlayer init
930 dfSerial.begin(9600, SERIAL_8N1, 16, 17);
931 if (!mp3.begin(dfSerial)) {
932     Serial.println("DFPlayer Mini not detected!");
933     lcdPrintLine(0, "MP3 Error");
934     lcdPrintLine(1, "");
935     while (true) delay(1000);
936 }
937 mp3.volume(g_volumeValue);
938 Serial.println("DFPlayer Mini initialized.");
939
940 // [C6] Flex thresholds + filters init
941 calculateFlexThresholds();
942 for (int i = 0; i < kFlexCount; i++) g_flexFilters[i].reset();
943
944 // [C7] Load gesture dictionary
945 parseGestureConfig(gestureConfigJson);
946
947 lcdPrintLine(0, "System Ready!");
948 lcdPrintLine(1, "Welcome ECO_Hands");
949 Serial.println("System ready.");
950
951 mp3.play(1);
952 delay(2000);
953 }
954
955 void loop() {
956     // [C10] Web UI service
957     server.handleClient();
958
959     const uint32_t now = millis();
960
961     // [C10] Reverse display window end
962     if (g_reverseDisplayed && (now - g_reverseStartMs >= REVERSE_DISPLAY_MS)) {
963         g_reverseDisplayed = false;
964     }
965
966     // [C6] Flex sensor read + preprocessing
967     // [C7] Gesture classification + action
968     if (now - g_prevFlexMs >= FLEX_INTERVAL_MS) {
969         g_prevFlexMs = now;
970
971         int flexPercent[kFlexCount];
972         for (int i = 0; i < kFlexCount; i++) {
973             int raw = analogRead(kFlexPins[i]);
974
975             long mapped = map(raw, g_flexMin[i], g_flexMax[i], 0, 100);
976             mapped = constrain(mapped, 0, 100);
977
978             flexPercent[i] = g_flexFilters[i].add((int)mapped);
979         }
980
981         classifyFingerStates(flexPercent);
982         recognizeGestureFromJSON();
983     }

```

```

984 // [C2] Acquire VAD audio chunk (I2S)
985 size_t bytesRead = 0;
986 i2s_read(I2S_NUM_0, (void*)g_vadPcm, CHUNK_BYTES, &bytesRead, portMAX_DELAY);
987
988 if (bytesRead > sizeof(g_pcmPayload)) bytesRead = sizeof(g_pcmPayload);
989 memcpy(g_pcmPayload, (const uint8_t*)g_vadPcm, bytesRead);
990
991 // [C8] VAD decision; [C9] If triggered, record + upload to Azure + parse
992 transcript
993 if (detectVoice(g_vadPcm, bytesRead / sizeof(int16_t))) {
994     Serial.println("Voice detected. Recording + sending...");
995     lcdPrintLine(1, "Voice Detected");
996     recordAndSendAudio(g_pcmPayload, bytesRead);
997 }
998
999 // [C10] Azure display management
1000 if (g_displayingAzure) {
1001     if (millis() - g_azureStartMs >= AZURE_DISPLAY_MS) {
1002         g_displayingAzure = false;
1003         g_scrolling = false;
1004         g_scrollText = "";
1005         lcdPrintLine(1, "");
1006     } else if (g_scrolling) {
1007         updateScroll();
1008     }
1009 }
1010 }

```

References

1. World Federation of the Deaf. (n.d.). FAQs. <https://wfdeaf.org/contact/faqs/>
2. World Health Organization. (2025, Feb 26). Deafness and hearing loss. <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>
3. Chong, T.-W., & Lee, B.-G. (2018). American Sign Language recognition using Leap Motion Controller with machine learning approach. *Sensors*, 18(10), 3554. <https://doi.org/10.3390/s18103554>
4. Chuan, C.-H., Regina, E., & Guardino, C. (2014). American Sign Language recognition using a Leap Motion sensor. In *Proceedings of the 13th IEEE International Conference on Machine Learning and Applications*. <https://doi.org/10.1109/ICMLA.2014.110>
5. Aryanie, D., & Heryadi, Y. (2017). Alphabet sign language recognition using Leap Motion technology and RBBPGANN (preprint). *ResearchGate*. <https://www.researchgate.net/publication/314097642>
6. MedCrave. (2022, Dec 22). Smart glove for sign language translation. *International Robotics & Automation Journal*. <https://medcraveonline.com/IRATJ/smart-glove-for-sign-language-translation.html>
7. Spectra Symbol. (2019). *Flex Sensor—Datasheet (v2019a)*. <https://spectra-symbol-landing.s3.us-west-1.amazonaws.com/data-sheets/Flex-Sensor-Datasheet-v2019a.pdf>
8. SparkFun. (n.d.). Flex Sensor Hookup Guide. <https://learn.sparkfun.com/tutorials/flex-sensor-hookup-guide/all>
9. SparkFun. (n.d.). Voltage dividers. <https://learn.sparkfun.com/tutorials/voltage-dividers/all>
10. Analog Devices. (2022). ADXL345: 3-Axis, ± 2 g/ ± 4 g/ ± 8 g/ ± 16 g Digital Accelerometer (Datasheet). <https://www.analog.com/media/en/technical-documentation/data-sheets/adxl345.pdf>
11. bestpika. (2017, June 13). ADC2 Channel cannot be used when WiFi is in use #440 [Issue]. GitHub. Retrieved January 2, 2026, from <https://github.com/espressif/arduino-esp32/issues/440>
12. DFRobot. (n.d.). DFPlayer Mini MP3 Player (SKU: DFR0299). DFRobot Wiki. Retrieved January 2, 2026, from https://wiki.dfrobot.com/dfplayer_mini_sku_dfr0299
13. Espressif Systems. (n.d.). Analog to Digital Converter (ADC). In *ESP-IDF Programming Guide (v4.0.3)*. Retrieved January 2, 2026, from <https://docs.espressif.com/projects/esp-idf/en/v4.0.3/api-reference/peripherals/adc.html>

14. Espressif Systems. (n.d.). *Analog to Digital Converter (ADC)*. In *ESP-IDF Programming Guide (v4.4.8)*. Retrieved January 2, 2026, from <https://docs.espressif.com/projects/esp-idf/en/v4.4.8/esp32/api-reference/peripherals/adc.html>
15. Espressif Systems. (n.d.). *ESP32-WROVER-E & ESP32-WROVER-IE datasheet (Version 2.0)* [Data sheet]. Retrieved January 2, 2026, from https://www.espressif.com/sites/default/files/documentation/esp32-wrover-e_esp32-wrover-ie_datasheet_en.pdf (Alternate access copy used during drafting: https://www.mouser.com/datasheet/3/1574/1/esp32-wrover-e_esp32-wrover-ie_datasheet_en.pdf)
16. ESP_igrr. (2022, October 16). *ADC2 and Wifi together — any hope?* [Online forum post]. ESP32 Forum. Retrieved January 2, 2026, from <https://esp32.com/viewtopic.php?t=30102>
17. InvenSense Inc. (2013, August 19). *MPU-6000 and MPU-6050 register map and descriptions (RM-MPU-6000A-00, Rev. 4.2)* [PDF]. Retrieved January 2, 2026, from <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf>
18. InvenSense Inc. (2013, August 19). *MPU-6000/MPU-6050 product specification (PS-MPU-6000A-00, Rev. 3.4)* [PDF]. Retrieved January 2, 2026, from <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
19. InvenSense Inc. (2014, May 21). *INMP441 omnidirectional microphone with bottom port and I2S digital output (DS-INMP441-00, Rev. 1.1)* [Data sheet]. Retrieved January 2, 2026, from <https://invensense.tdk.com/wp-content/uploads/2015/02/INMP441.pdf>
20. Microsoft. (2025, November 21). *Use speech to text REST API for short audio*. Microsoft Learn. Retrieved January 2, 2026, from <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/rest-speech-to-text-short>
21. Spectra Symbol. (2010). *Flex sensor data sheet '10 (Rev. A2)* [Data sheet]. Retrieved January 2, 2026, from <https://cdn.sparkfun.com/assets/8/e/7/a/0/flex22.pdf>
22. Texas Instruments. (2023). *LM2596 SIMPLE SWITCHER® 150-kHz 3-A step-down voltage regulator (Rev. G)* [Data sheet]. Retrieved January 2, 2026, from <https://www.ti.com/lit/ds/symlink/lm2596.pdf>
23. Zaitronics. (2025, December 24). *How to use 16x2 LCD (Parallel & I2C)*. Zaitronics. Retrieved January 2, 2026, from <https://zaitronics.com.au/blogs/guides/how-to-use-16x2-lcd-parallel-i2c>
24. All About Circuits. (2021, Dec 12). *Introduction to capacitive accelerometers*. <https://www.allaboutcircuits.com/technical-articles/introduction-to-capacitive-accelerometer-measure-acceleration-capacitive-sensing/>
25. Analog Devices. (n.d.). *Accelerometer (glossary entry)*. <https://www.analog.com/en/resources/glossary/accelerometer.html>
26. TDK InvenSense. (2013). *INMP441: Omnidirectional MEMS microphone with I²S output (Datasheet)*. <https://invensense.tdk.com/wp-content/uploads/2015/02/INMP441.pdf>
27. Microsoft. (2021, Jan 7). *Resource Interchange File Format (RIFF)*. <https://learn.microsoft.com/windows/win32/xaudio2/resource-interchange-file-format--riff->
28. McGill University. (2022, Sep 27). *Wave file specifications*. <https://www.mmsp.ece.mcgill.ca/Documents/AudioFormats/WAVE/WAVE.html>
29. soundfile.sapp.org. (n.d.). *WAVE PCM soundfile format*. <https://soundfile.sapp.org/doc/WaveFormat/>
30. Microsoft Learn. (2025, May 25). *Speech to text REST API for short audio*. <https://learn.microsoft.com/azure/ai-services/speech-service/rest-speech-to-text-short>
31. Microsoft Learn. (2025, Apr 28). *Audio concepts in Azure AI Speech*. <https://learn.microsoft.com/azure/ai-services/speech-service/concepts/audio-concepts>
32. Microsoft Learn. (2025, May 25). *Speech to text REST API (overview)*. <https://learn.microsoft.com/azure/ai-services/speech-service/rest-speech-to-text>
33. Arduino Docs. (2025, Jun 20). *ArduinoJson (library page)*. <https://docs.arduino.cc/libraries/arduinojson>
34. Blanchon, B. (n.d.). *ArduinoJson v6 documentation*. <https://arduinojson.org/v6/doc/>
35. Espressif. (n.d.). *Preferences—Arduino-ESP32 documentation*. <https://docs.espressif.com/projects/arduino-esp32/en/latest/tutorials/preferences.html>
36. Espressif. (n.d.). *Wi-Fi API (softAP)*. <https://espressif-docs.readthedocs-hosted.com/projects/arduino-esp32/en/latest/api/wifi.html>
37. Random Nerd Tutorials. (2018). *ESP32 Access Point (AP) Web Server*. <https://randomnerdtutorials.com/esp32-access-point-ap-web-server/>
38. Random Nerd Tutorials. (2020). *ESP32 HTTP GET and POST (HTTPClient)*. <https://randomnerdtutorials.com/esp32-http-get-post-arduino/>
39. Rickman, J. (n.d.). *LiquidCrystal_I2C (Arduino LCD I²C library)* [GitHub repository]. https://github.com/johnrickman/LiquidCrystal_I2C